
CommerceCore: Low-Budget Multitask Adaptation for Structured E-Commerce Queries

Arghya Mukherjee
Independent Researcher
arghya05@gmail.com
ORCID: 0009-0008-3423-8574

Abstract

Structured shopping-query extraction requires both schema compliance and faithful interpretation of user constraints. We study CommerceCore, a 1.7-billion-parameter Qwen3 model adapted with QLoRA using 1,500 human-annotated QueryNER examples, 450 synthetic query examples, and 180 synthetic catalog examples. The recorded training run takes 75.2 minutes, corresponding to an estimated GPU charge of \$0.90 at the recorded hourly rate. The selected checkpoint has a recorded mean per-query entity-set F1 of 0.498 on 993 QueryNER examples and constraint-set F1 of 0.725 on 20 development queries. These scores require qualification: 30 QueryNER test examples informed checkpoint selection, and available API baselines use a different sample and an incomplete label vocabulary. An offline audit identifies 374 distinct synthetic queries and eight query groups with conflicting target sets. Reanalysis of six diagnostic predictions gives mean F1 of 0.400, micro-F1 of 0.381, and no exact-set matches, exposing failures in completeness, negation, and normalization. The evidence supports low-budget implementation feasibility, but does not establish superiority over general-purpose models or production reliability. We provide a traceable account of the implementation, distinguish recorded measurements from reproducible offline analyses, and identify the conditions required to test stronger claims.

1 Introduction

Shopping queries combine product descriptions with constraints on attributes, price, and usage. A query such as “black waterproof trainers under 80 pounds” requires identifying a color, interpreting an attribute as Boolean, and mapping a price phrase to a catalog field. A syntactically valid JSON response can still omit a requirement or reverse its meaning. When search systems execute these constraints, such errors change the products a user sees.

A small generative model offers a practical interface: the same backbone can produce structured outputs for query segmentation, constraint extraction, and catalog attributes. Its usefulness depends on more than parameter count. Training targets must consistently cover what the input expresses, and evaluation must provide comparable evidence to every system. The central research question is therefore: *under what conditions does task specialization provide an effective and economical model for structured commerce queries?*

We investigate this question through CommerceCore, a Qwen3-1.7B model [6, 7] adapted on real and synthetic examples. We examine three narrower questions supported by the available artifacts. **RQ1:** What resource use and task behavior are recorded for this adaptation? **RQ2:** Do the data and evaluation protocols support comparative accuracy claims? **RQ3:** What do checkpoint trajectories and saved predictions reveal about its limits? The broader effects of data composition, out-of-distribution robustness, and deployment configuration require additional controlled experiments.

The recorded run demonstrates a small incremental training budget. However, auditing the artifacts changes the interpretation of its accuracy results: the constraint set was used throughout development, part of the QueryNER test set informed selection, and the available QueryNER API prompt names only two of 17 entity types. Mechanically accepted synthetic examples also contain conflicting targets. These issues affect the inference drawn from a score even when its arithmetic is correct.

Our contributions are: (1) an implementation-level account of three-task adaptation, including the data mixture, objective, checkpoint history, and resource use; (2) a reproducible audit of target consistency, query overlap, baseline label coverage, and selection exposure; and (3) case-level analysis of all six saved diagnostic predictions, including micro-F1 and exact-set accuracy. We use established adaptation methods and make no novel-architecture or state-of-the-art claim. The study connects a concrete low-budget implementation to the conclusions its evidence can support.

2 Related Work

Commerce query understanding. QueryNER provides human-annotated shopping queries with 17 entity types and evaluates supervised BERT and XLM-R taggers, including domain-adapted BERT [1]. These are relevant specialist baselines for a generative segmentation model. CommerceCore serializes label-text pairs as JSON; its per-query set metric is not interchangeable with token-indexed sequence-labeling evaluation. We make no leaderboard comparison against the original QueryNER results.

Small-model specialization. Licardo and Tankovic [2] study QLoRA adaptation of a one-billion-parameter Llama 3.2 model for multilingual commerce intent recognition and deployment quantization. This is close prior art for the small-model specialization strategy. CommerceCore instead produces constraint and entity sets, includes human-annotated data, and examines label and evaluation integrity. Neither changing the backbone nor using synthetic data establishes methodological novelty.

Multitask adaptation. EcomGPT [3] studies commerce instruction tuning using EcomInstruct and a chain-of-task formulation. CommerceCore uses a much smaller mixture with plain task prefixes, without chain-of-task supervision. LoRA learns low-rank updates to frozen weights [4]; QLoRA combines such updates with a quantized backbone [5]. We adopt this established recipe. The catalog task is auxiliary and lacks a separate evaluation, so we cannot attribute gains to sharing tasks rather than training specialists.

3 CommerceCore: Tasks, Data, and Adaptation

3.1 Output contracts and training mixture

For a query x , constraint extraction predicts $C(x) = \{(f_j, o_j, v_j)\}_{j=1}^m$, where f_j is a schema field, o_j a comparison operator, and v_j a string-valued canonical value. Synthetic targets cover nine fields: color, material, size, waterproof status, added-sugar presence, nut presence, price in GBP, power source, and warranty duration in months. Boolean values are strings. Historical labels map “under 80” to `lte`; scoring follows that convention without claiming strict linguistic equivalence.

Entity segmentation predicts (ℓ, s) pairs, with QueryNER label ℓ and span text s . Local records preserve tokens and half-open token offsets; training converts spans to space-joined text. Catalog extraction maps a listing to an attribute dictionary. Thus three tasks are trained, while only two have task-level evaluation artifacts.

The QueryNER builder shuffles 7,841 local training records with seed 42 and selects 1,500 (Table 1). The local test file contains 993 records. Both identify source revision `f2f9173d14243496dbcf017a9dc2ded74989a45e`. We report counts from these files rather than assuming all published versions have identical counts. The dataset card declares CC BY 4.0 [8], and local provenance identifies Amazon ESCI as the source of the underlying queries. An offline audit finds no normalized exact query overlap between the local train and test files; this does not exclude paraphrase overlap or pretraining contamination.

Table 1: Training mixture verified against the archived 2,130-row file. Rebuilding with seed 42 reproduces the archived prompt/completion records in order.

Task	Source	Rows	Share
Entity segmentation	QueryNER, human annotated	1,500	70.4%
Query constraints	Synthetic, 150 per vertical	450	21.1%
Catalog attributes	Synthetic, 60 per vertical	180	8.5%
Total		2,130	100.0%

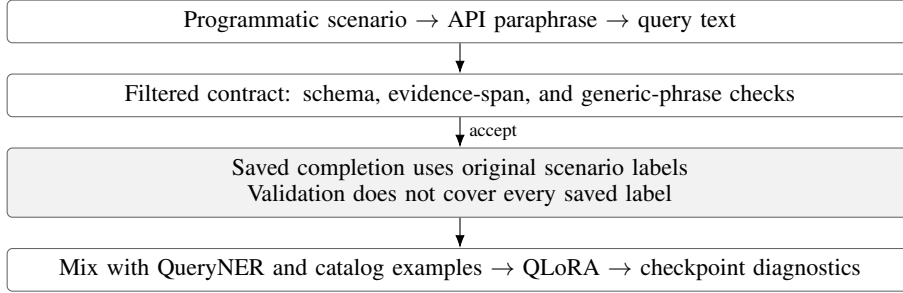


Figure 1: The implemented query-data path. The discrepancy between the validated contract and the saved target is an observed implementation property.

3.2 Synthetic generation and validation

The query generator samples a structured scenario in code and asks an API model, identified as `claude-sonnet-4-5`, to render it as a shopping query. Query and catalog data cover fashion, grocery, and general merchandise equally. Generated targets constitute weak supervision, not new human annotation.

The query path constructs an intermediate contract by finding literal value substrings or predefined Boolean surface forms, omitting constraints with no match. A validator checks schema structure, span binding, and a blocklist of generic phrases. Although the validator supports simulated execution, the scaled query-generation call supplies neither tool calls nor a catalog; execution correctness is not checked on that path.

The writer then serializes the *original scenario constraints*, rather than the filtered contract that passed validation (Figure 1). Acceptance of that contract therefore does not guarantee grounding of every saved target. Substring binding also cannot prove completeness or semantic fidelity. For example, finding an attribute word does not by itself resolve whether it is negated.

3.3 Training configuration

The base is Qwen3-1.7B, distributed under Apache 2.0 [7]. Training uses 4-bit NF4 with double quantization and bfloat16 computation; LoRA rank 16, alpha 32, and dropout 0.05; and adapters on the attention query/key/value/output and feed-forward gate/up/down projections. Gradient checkpointing is enabled. For prompt p_i and completion y_i , training minimizes completion-token negative log likelihood:

$$\mathcal{L}(\theta) = - \sum_i \sum_{t \in y_i} \log p_{\theta}(y_{i,t} \mid p_i, y_{i,<t}). \quad (1)$$

Prompt tokens are masked, an end-of-sequence token is appended, and the concatenated sequence is truncated to 512 tokens. AdamW uses learning rate 10^{-4} , microbatch one, and eight accumulation steps. The once-shuffled mixture is cycled sequentially without task reweighting or epoch reshuffling. The training seed is 42. There are 1,200 optimizer steps, with evaluation every 150 steps, corresponding to approximately 4.51 passes through 2,130 rows. The selected step 750 represents 2.82 passes. Checkpoint task decoding is greedy with 100 new tokens, plain prefixes, and no constrained JSON decoder.

4 Experimental Setup and Evidence Audit

4.1 Metrics and parsing

Historical scores are means of *per-query exact-set F1*, not label-macro-F1. For predicted set P_i and gold set G_i ,

$$F_{1,i} = \frac{2|P_i \cap G_i|}{|P_i| + |G_i|}, \quad \bar{F}_1 = \frac{1}{n} \sum_{i=1}^n F_{1,i}. \quad (2)$$

Two empty sets score one. Constraints match exact field/operator strings and stringified values. Entity matching uses the exact label and lowercased, stripped span text; token offsets are discarded. Set conversion removes duplicates, so repeated identical predictions are not penalized and identical span text at different positions is indistinguishable.

The parser extracts a JSON list with a regular expression; missing or invalid JSON becomes an empty list. This conflates parse failure with a valid empty prediction. One of the 20 constraint development queries has empty gold, giving an always-empty system mean F1 of 0.050. For the six saved diagnostic predictions, we additionally compute pooled precision, recall, micro-F1, and exact-set accuracy $n^{-1} \sum_i \mathbf{1}[P_i = G_i]$.

4.2 Data splits and selection exposure

The 20 author-written constraint queries comprise seven fashion, seven grocery, and six general-merchandise examples. Despite the filename `held_out_eval_set.jsonl`, all 20 are evaluated at every checkpoint. We therefore treat them as *development data*. Their normalized query strings do not exactly match the synthetic training file, but they are not an untouched final evaluation.

The training script likewise evaluates the first 30 QueryNER test records at each checkpoint. The later 993-example score includes these records and is selection-exposed. Full-test per-example predictions are absent, preventing rescoring on the remaining 963 records or a paired comparison. Retrospective removal would also not recreate a prospectively frozen evaluation.

Six additional author-written queries provide diagnostics for composition, negation, permissive language, and unit/vocabulary normalization. Their strings do not exactly match the current synthetic file or development set. They remain a small convenience sample with schema-specific, sometimes incomplete gold labels; they are not a representative out-of-distribution benchmark.

4.3 Baseline comparability

The QueryNER API script evaluates the *first 50* test examples and requests only `core_product_type` or `modifier`. On that sample, 52 of 119 annotated spans have other labels. Let k_i denote the number of gold label–text pairs allowed by the prompt and g_i the total number of gold pairs. A perfect predictor restricted to the allowed labels obtains

$$F_{1,i}^{\text{restricted}} = \frac{2k_i}{k_i + g_i}. \quad (3)$$

The optimal prediction includes every permitted gold pair and no incorrect pair, since incorrect additions increase only the denominator. Averaging this bound over the 50 examples gives 0.648. This is a prompt-compliance ceiling, not a bound on a model that ignores the instruction. Different samples and label vocabularies preclude interpreting the available scores as evidence of CommerceCore’s superiority.

The constraint API scripts use two demonstrations but do not specify the full field vocabulary. Historical IDs are `claude-haiku-4-5-20251001`, `claude-sonnet-5`, `gpt-4o-mini`, and `gpt-5`; these are recorded identifiers, not claims about current availability or provider tiers. Their numerical aggregates are preserved in Appendix A. Per-request metadata and raw generations are incomplete.

The open-model script uses Qwen3.5-2B and SmolLM3-3B with plain text prompts and 150 new tokens, without model-specific chat templates or explicit reasoning-mode settings. Both have empty parsed predictions on all six diagnostic queries. Without raw generations, empty answers cannot be distinguished from truncation or parser failure. These comparisons do not isolate fine-tuning or scale.

Table 2: CommerceCore results and evidence scope. These evaluations use different example populations and do not constitute interchangeable test estimates.

Evaluation	n	$\bar{F}_1$	Evidence
Constraint development, step 750	20	0.725	Recorded checkpoint aggregate
Served-model development	20	0.683	Recomputed from predictions
QueryNER selection subset, step 750	30	0.556	Recorded checkpoint aggregate
QueryNER full local test file	993	0.498	Recorded final aggregate
Additional diagnostic queries	6	0.400	Recomputed from predictions

4.4 Uncertainty and provenance

The full QueryNER file records mean F1 0.498474 and a 10,000-resample percentile interval [0.474948, 0.522320]. We retain this as a *recorded interval*, since its score vector is absent. It describes query resampling under the implemented metric, not training-seed variation, selection bias, or prompt uncertainty. The repeated constraint-baseline script instead bootstraps 60 entries from three calls on 20 queries. Those entries are not independent queries; valid query-level inference would cluster repeats and preferably estimate paired differences. We therefore do not use its pooled intervals for comparative inference or infer population uncertainty from six deliberately chosen examples.

5 Results and Analysis

5.1 Task performance and training-target consistency

Table 2 reports the supported task results. “Recorded” denotes an available aggregate; “recomputed” denotes rescored saved predictions. The full-file QueryNER result is lower than the 30-example selection-subset score, but different example populations and evaluation stages prevent attributing the gap to one cause. The 20-query score is developmental and cannot certify final generalization.

The offline audit finds 374 distinct normalized queries in 450 synthetic rows, with 76 repeated rows beyond the first occurrence. Eight query groups, containing 20 rows, have conflicting target *sets*; list-order differences are ignored. For example, “black waterproof windbreaker size S” is paired with color alone in one row and color plus waterproof status in another. The prompt supplies no scenario identifier that could distinguish these targets. Target lengths are one constraint in 192 rows, two in 238, and three in 20; none contains four. No current query row matches the generic-phrase blocklist. This verifies phrase removal, not linguistic naturalness or semantic fidelity. Historical labels are preserved so that edits do not silently sever their connection to the recorded model.

5.2 Benchmark results by evaluation protocol

The benchmark tables retain the original study’s entity-segmentation, constraint-extraction, and harder-query evaluations. We separate their sample sizes and execution protocols so that the recorded scores can be read in context. Values are rounded to three decimals from the saved artifacts; these tables do not represent new model runs. Additional values reported in the original manuscript, for which a matching measurement file is unavailable, are preserved separately in Appendix A.

Table 3: QueryNER entity-set F1. The CommerceCore evaluation uses 993 examples, including the 30 used during checkpoint selection. API runs use only the first 50 examples and a two-label prompt. The rows are not a matched comparison.

System	Queries	Mean F1	Evaluation scope
CommerceCore, step 750	993	0.498	Full local test file; recorded 95% interval [0.475, 0.522].
Claude Haiku 4.5	50	0.263	First 50; two permitted output labels.
GPT-4o mini	50	0.294	First 50; same restricted label prompt.

Table 4: Constraint-extraction mean F1 on the 20 development queries. API rows are three-repeat aggregates (60 scores per system); local-model rows are single evaluations. Prompt, decoding, and selection differences prevent attributing score differences to model specialization.

System	Mean F1	Recorded protocol
CommerceCore, step 750	0.725	Training-time checkpoint evaluation; greedy decoding.
CommerceCore, served model	0.683	Merged-model predictions rescored from the archive.
Qwen3.5-2B, unfine-tuned	0.050	Archived few-shot aggregate; no per-query predictions.
SmolLM3-3B, unfine-tuned	0.100	Archived few-shot aggregate; no per-query predictions.
Claude Haiku 4.5	0.518	Three few-shot API runs.
Claude Sonnet 5	0.592	Three few-shot API runs; corrected stored aggregate.
GPT-4o mini	0.618	Three few-shot API runs.
GPT-5	0.504	Three few-shot API runs.

The served-model archive contains all 20 predictions and labels. Rescoring reproduces mean F1 0.683, with 23 matching triples out of 31 predictions and 35 gold triples (micro-F1 0.697), and seven exact-set matches (35%). This is the original paper’s served-model benchmark. Its difference from checkpoint F1 0.725 does not isolate an effect of merging, precision, or serving because the evaluation stages differ. The archived open-model results restore the original 0.050 and 0.100 scores under their recorded plain-prompt protocol.

Table 5: All seven systems from the six-query diagnostic comparison, retaining the original harder-query results. API values are stored aggregates; local-model values are recomputed from saved parsed predictions. Provider-side prompts and raw generations are incomplete, so this is descriptive evidence on six cases.

System	Mean F1	Available evidence
CommerceCore (1.7B)	0.400	Parsed predictions; 0/6 exact-set matches.
Qwen3.5-2B, unfine-tuned	0.000	Six empty parsed outputs.
SmolLM3-3B, unfine-tuned	0.000	Six empty parsed outputs.
Claude Haiku 4.5	0.298	Aggregate; no saved per-query API responses.
Claude Sonnet 5	0.493	Aggregate; no saved per-query API responses.
GPT cheap tier	0.428	Aggregate; tier alias retained from the file.
GPT flagship	0.167	Aggregate; tier alias retained from the file.

The stored diagnostic means for Sonnet and the GPT cheap-tier configuration are above CommerceCore’s 0.400. The sample is too small and incompletely documented to establish a general ranking. Likewise, the two open-model zeros describe empty parsed outputs under the saved prompting and parsing configuration, rather than establishing the models’ capability under appropriate templates and generation budgets.

5.3 Checkpoint selection and behavioral drift

At each checkpoint, the model answers five fixed prompts about a capital city, weather, arithmetic, a primary color, and a sentence completion. Let A and B be sets of lowercased whitespace-delimited output tokens from the checkpoint and frozen base. The diagnostic averages

$$S(A, B) = \frac{|A \cap B|}{\max(|A|, |B|)}, \quad (4)$$

with one for two empty sets and zero if only one is empty. It measures lexical agreement: a correct paraphrase can lower it, while a shared wrong answer can raise it. It is not a validated measure of retained general capability.

The historical selection note chooses step 750, where output similarity is highest (0.6143), constraint F1 is 0.7250, and QueryNER subset F1 is 0.5562. At step 1,200 these are 0.4886, 0.7400, and 0.5298. Step 600 maximizes QueryNER subset F1 and step 1,050 maximizes constraint F1 (Figure 2). Pareto analysis over all three logged signals retains steps 150, 300, 600, 750, 900, and 1,050. Step 750 is one nondominated choice, not a uniquely optimal checkpoint. It was selected after observing

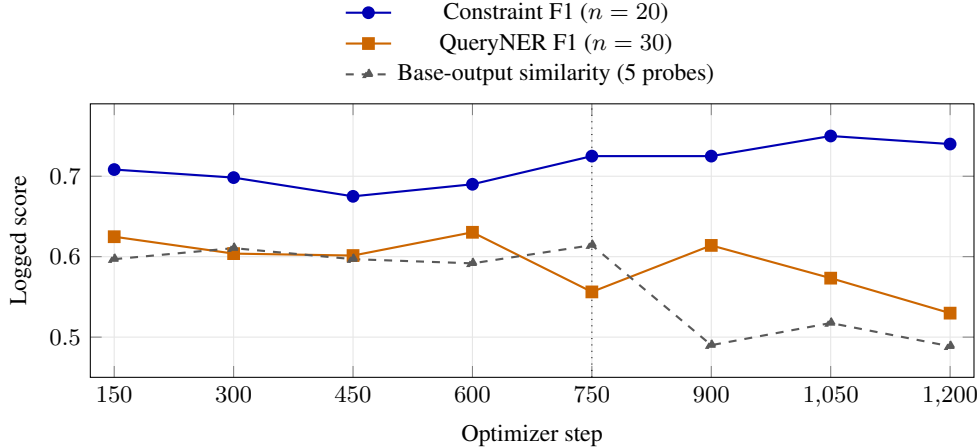


Figure 2: All eight recorded checkpoints. The dotted line marks selected step 750. Similarity and extraction F1 are different quantities despite sharing a plotting scale.

Table 6: All six CommerceCore diagnostic cases. Prediction fields are abbreviated here; rescoring uses the exact stored triples.

Query	Saved prediction	F1	Observation
green cotton hoodie size L under 40 pounds	price $\leq$ 40	0.400	Omits color, material, size.
waterproof boots not black	waterproof = true; color = "not black"	0.667	Literalized negation; gold omits negative color.
protein bar no nuts please, sugar is fine	nuts = false; added sugar = true	0.667	Turns permission into a requirement.
vegan chocolate with no added sugar under 5 pounds	price $\leq$ 5	0.667	Omits no-added-sugar; vegan absent from gold.
portable speaker under 3 year warranty running on batteries	warranty $\geq$ 3 months; price $\leq$ 3	0.000	Field/operator/unit errors; missing power.
wall powered toaster at least 18 months guarantee	power source = "wall"	0.000	Noncanonical value; missing warranty.

the trajectories, without a prespecified scalar objective or threshold. The later similarity decline establishes output drift on these probes, not catastrophic forgetting or a causal effect of overtraining.

The complete 1,200-step loss trace, approximate epoch summaries, and exact losses at all eight checkpoints are included in Appendix B. The mean logged loss decreases from 0.2136 over steps 1–266 to 0.0246 over the final partial pass (steps 1,066–1,200). These are training-loss summaries, not held-out loss estimates; the decline does not establish better extraction accuracy or preserved general capability.

5.4 Diagnostic errors

The six saved predictions contain nine predicted triples and twelve gold triples, with four matches. Pooled precision is 0.444, recall is 0.333, and micro-F1 is 0.381. Mean per-query F1 is 0.400, but exact-set accuracy is 0/6. Partial overlap therefore coexists with an incorrect constraint set on every example. Table 6 reports every case, including the distinction between permitting sugar and requiring its presence, and errors in field, operator, and unit normalization.

Gold-label scope also limits interpretation. The boots example has only a waterproof target, omitting the negative color condition even though the broader schema supports neq. "Vegan" is absent from both the chocolate target and the empty development oat-milk target. Scores measure agreement with these restricted annotations, not complete shopping-query understanding. These cases illustrate errors but cannot establish their prevalence.

6 Efficiency, Limitations, and Implications

Recorded compute and cost. The run summary records 4,510.6 seconds (75.18 minutes); at the encoded historical rate of \$0.72 per GPU hour, the estimated charge is \$0.9021. The timer includes periodic evaluation and checkpointing but excludes earlier loading and base-probe generation. Peak `torch.cuda.max_memory_allocated` is recorded as 2.13 GiB, which excludes reserved and non-PyTorch device memory. It is not a minimum hardware requirement. The project identifies an RTX PRO 4500 with 32 GB; the summary does not embed a hardware trace.

The cumulative generation ledger records 1,630 calls and an estimated \$1.88 at its encoded token prices. This is generation history, not a complete project budget. Evaluation APIs, startup, idle time, storage, and engineering effort are not fully reconciled. No sustained-load serving trace is available for independently auditing latency percentiles or throughput. For hourly rate r , sustained throughput q , and utilization u , compute-only cost per query is $r/(uq)$. Batching capacity and utilization cannot be inferred from a few request timings; no measured superiority in serving economics is claimed.

Causal and comparative limitations. The run supports implementation feasibility (RQ1). It lacks a matched unfine-tuned Qwen3-1.7B evaluation, supervised encoder baseline, deterministic baseline, and controlled single-task or data-source ablations. The existing comparisons do not establish superiority (RQ2), and the three synthetic verticals are not independent merchant populations. Only one training seed is available. Neither the contribution of catalog supervision nor positive task transfer is measured.

Supervision and robustness. The writer/validator discrepancy and conflicting targets show that scenario-first generation alone does not ensure faithful labels. Incomplete predictions are consistent with incomplete supervision, but a causal test requires retraining on repaired data under a matched protocol. The catalog targets also need their own fidelity audit. Five output-similarity probes and six diagnostic queries identify behaviors (RQ3), not general-capability retention or population robustness. A confirmatory design is specified in Appendix D; its experiments are not reported as completed.

Practical use and broader impact. The service is a research interface. Valid JSON can contain unsupported or conflicting constraints, and empty lists can mask parsing failures. Execution should distinguish those states and preserve user intent, particularly for dietary or allergen requirements that these results cannot certify. English queries and narrow synthetic schemas limit representativeness. Self-hosting can provide architectural control over data flow, but no privacy guarantee or security audit is established here. Dataset attribution and underlying query provenance should accompany redistribution; synthetic labels should remain distinguishable from human annotation.

7 Reproducibility

The public artifacts are the [GitHub repository](#), [Hugging Face model card](#), and [versioned model files and archive](#). The latter includes merged weights, tokenizer files, checkpoint adapters, training data, logs, and evaluation records. Availability does not by itself verify that the released weights reproduce every recorded run. Appendix C provides clickable evidence links, fixed repository revisions, and instructions for checking the numbers.

The standard-library script `paper/audit_evidence.py` recounts rows, reproduces the archived seed-42 mixture in order, checks exact overlaps and conflicting target sets, computes the restricted-label ceiling, rescues the 20 served-model and six diagnostic predictions, and regenerates loss and checkpoint data. It saves [machine-readable results](#), reconstructed sample IDs, and SHA-256 hashes. Small archive inputs and the historical mixture are mirrored under `paper/evidence/archive/` so the audit can run from the GitHub checkout without downloading model weights. Full-test per-example QueryNER generations and a complete pinned training environment remain unavailable, and several original inference scripts retain pod-specific paths. These offline checks are distinct from rerunning model inference.

8 Conclusion

CommerceCore is a concrete case study of low-budget multitask adaptation for structured shopping queries. Its recorded run supports implementation feasibility, while saved predictions expose semantic errors despite partial set overlap. Auditing the artifacts identifies conflicting synthetic targets, test examples used for checkpoint selection, and an incomplete baseline label vocabulary. These findings constrain the claims supported by the current scores and provide executable checks for future studies. Strong comparative conclusions require consistent supervision, matched baselines, and an independent final evaluation.

Acknowledgments and Disclosure of Funding

The project used Anthropic and OpenAI models for API comparisons and AI assistance. Synthetic query generation names an Anthropic model in its script. AI assistance was used in manuscript preparation and artifact review. This disclosure does not substitute for verification of labels, measurements, or citations. No complete funding or expenditure record is established by the local artifacts.

References

- [1] Chester Palen-Michel, Lizzie Liang, Zhe Wu, and Constantine Lignos. 2024. [QueryNER: Segmentation of E-commerce Queries](#). In *Proceedings of LREC-COLING*, pp. 13455–13470.
- [2] Josip Tomo Licardo and Nikola Tankovic. 2025. [Performance Trade-offs of Optimizing Small Language Models for E-Commerce](#). arXiv:2510.21970.
- [3] Yangning Li, Shirong Ma, Xiaobin Wang, Shen Huang, Chengyue Jiang, Hai-Tao Zheng, Pengjun Xie, Fei Huang, and Yong Jiang. 2023. [EcomGPT: Instruction-tuning Large Language Models with Chain-of-Task Tasks for E-commerce](#). arXiv:2308.06966.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-Rank Adaptation of Large Language Models](#). In *International Conference on Learning Representations*.
- [5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. [QLoRA: Efficient Finetuning of Quantized LLMs](#). In *Advances in Neural Information Processing Systems*, vol. 36.
- [6] Qwen Team. 2025. [Qwen3 Technical Report](#). arXiv:2505.09388.
- [7] Qwen Team. 2025. [Qwen3-1.7B model card](#). Hugging Face. Accessed September 27, 2026.
- [8] Broadening Linguistic Technologies Lab. 2024. [QueryNER dataset card](#). Hugging Face. Local source revision identified in Section 3.

A Additional Benchmark Details and Original Results

Tables 3–5 present the main saved benchmark results. Table 7 adds the constraint results by vertical: seven fashion, seven grocery, and six general-merchandise queries. CommerceCore is a single served-model evaluation; API means cover three calls per query. The overall mean weights queries equally rather than averaging the three vertical means. Historical notes describe the Sonnet repetitions as identical, but omitting a temperature parameter is not itself proof of deterministic generation.

The six-query API file contains aggregates and aliases, without per-query responses or the provider-side execution script. Its values cannot support paired significance testing or a response-level fairness audit. The GPU-side diagnostic file has parsed predictions and gold labels, permitting rescoring but not inspection of raw generations.

Table 7: Constraint F1 by vertical: CommerceCore is rescored from archived served-model predictions; API values come from the three-repeat aggregate file. These are development-set results under differing protocols.

Stored model / alias	Fashion	Grocery	General	Overall
CommerceCore, served model	0.690	0.667	0.694	0.683
Claude Haiku 4.5	0.803	0.448	0.267	0.518
Claude Sonnet 5	0.795	0.643	0.294	0.592
GPT-4o mini / cheap tier	0.875	0.605	0.333	0.618
GPT-5 / flagship	0.759	0.540	0.167	0.504

A separate single-run flagship artifact records 0.608 for the Sonnet configuration and 0.542 for GPT-5 on the same development set. These are distinct runs from the three-repeat aggregates. We preserve their files rather than merging unlike measurements. Historical scripts convert API errors into empty predictions or zeros, so end-to-end scores can include request failures.

Preserving the original manuscript’s numerical record. Table 8 retains the two remaining QueryNER values from the [original manuscript](#). A matching result file or prediction trace for these two values has not been located in the repository or the inspected model archive. Their presence here records what the original paper reported; it does not independently verify the measurements or establish comparability with the recorded runs above. The original served-model and open-model constraint results, in contrast, have archive evidence and are included in Table 4.

Table 8: Previously reported benchmark values without a matching local measurement artifact. All entries in this table have narrative-only provenance and are excluded from comparative conclusions. Model aliases follow the original paper.

System / evaluation	Reported F1	Scope reported in the original manuscript
GPT flagship / QueryNER	0.204	Entity segmentation; sample and execution details unverified.
Claude Sonnet 5 / QueryNER	0.427	Entity segmentation; sample and execution details unverified.

B Training Logs, Checkpoint History, and Reproducibility

B.1 Complete loss trace and approximate epoch summaries

The released clean-run log contains one loss record for every optimizer step from 1 to 1,200. At each step, the training script averages the completion-token losses of eight microbatches; the logged value is rounded to four decimal places. It is an average of microbatch losses, not a token-weighted average over an epoch. Figure 3 plots every logged value and a trailing mean over at most 50 steps. The original CSV is included unchanged in the source package as `evidence/training_loss_history.csv` and is available in the repository at [the complete training-loss log](#).

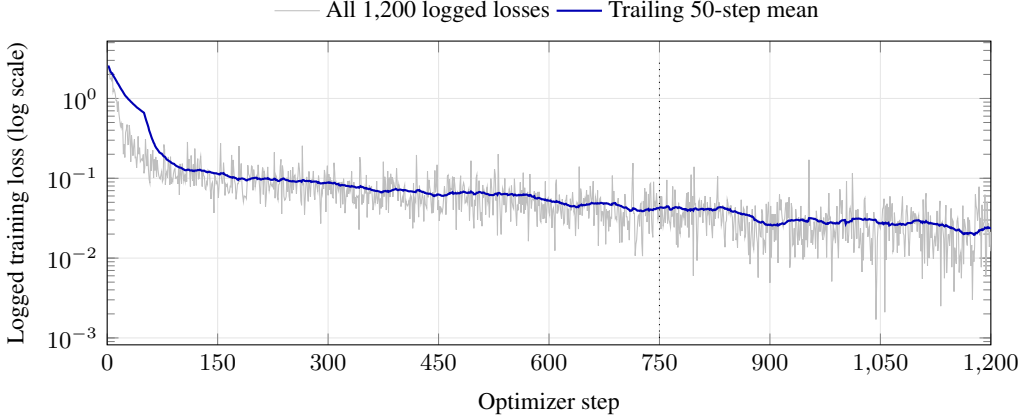


Figure 3: Training loss from the archived run, including individual-step fluctuations. The dotted line marks selected checkpoint 750. Smoothing summarizes the stored loss values and does not add observations; no validation-loss trace was recorded.

One pass through 2,130 examples spans 266.25 optimizer steps at accumulation eight. Table 9 uses boundaries $\lfloor k \cdot 2,130/8 \rfloor$, followed by the final partial pass. Some optimizer steps straddle pass boundaries, so these are approximate epoch windows. Means are arithmetic averages of the rounded logged values; the end-step column also preserves the epoch-boundary loss view used in the project record. Lower training loss alone does not establish improved generalization.

Table 9: Loss by approximate training pass, recomputed from all 1,200 records. The final row covers the incomplete fifth pass.

Steps	Passes at end	Logged steps	Mean loss	End-step loss
1–266	1.00	266	0.2136	0.0833
267–532	2.00	266	0.0698	0.0725
533–798	3.00	266	0.0461	0.0469
799–1,065	4.00	267	0.0303	0.0361
1,066–1,200	4.51	135	0.0246	0.0122

B.2 Every checkpoint with its exact training-loss record

Table 10: Complete eight-checkpoint history joined to the exact loss-log step. C-F1 uses 20 constraint development queries; QN-F1 uses the first 30 QueryNER test examples; similarity uses five lexical probes. Elapsed minutes are logged before that step’s checkpoint evaluation and saving. Step 750 is selected.

Step	Passes	Loss	C-F1	QN-F1	Similarity	Minutes
150	0.56	0.0836	0.7083	0.6248	0.5969	7.21
300	1.13	0.0612	0.6983	0.6038	0.6106	16.44
450	1.69	0.0530	0.6750	0.6014	0.5969	25.87
600	2.25	0.0374	0.6900	0.6303	0.5917	35.37
750	2.82	0.0489	0.7250	0.5562	0.6143	44.76
900	3.38	0.0049	0.7250	0.6140	0.4900	54.24
1,050	3.94	0.0234	0.7500	0.5733	0.5175	63.69
1,200	4.51	0.0122	0.7400	0.5298	0.4886	73.10

At step 1,200, the loss log shows 4,385.8 seconds (73.10 minutes), while the run summary records 4,510.6 seconds (75.18 minutes). The latter also includes the final checkpoint evaluation and saving. A step’s loss is the eight-microbatch mean for that update, not the mean loss over all preceding training. The table uses the actual checkpoint steps rather than nearby illustrative loss samples.

The local selected adapter configuration agrees with rank 16, alpha 32, dropout 0.05, and the seven projection targets. It identifies PEFT version 0.21.0 and a pod-local base path, but no base revision. This is not a complete environment lock. The training script uses standard AdamW rather than a paged optimizer.

The exact diagnostic prompts are: “What is the capital of France?”, “Write a short sentence about the weather today.”, “What is 12 plus 15?”, “Name one primary color.”, and “Complete this sentence: The sun rises in the”. Decoding is greedy with 30 new tokens. Only similarity against the frozen base is logged; assigning a variable for the preceding checkpoint does not create an additional retention measurement.

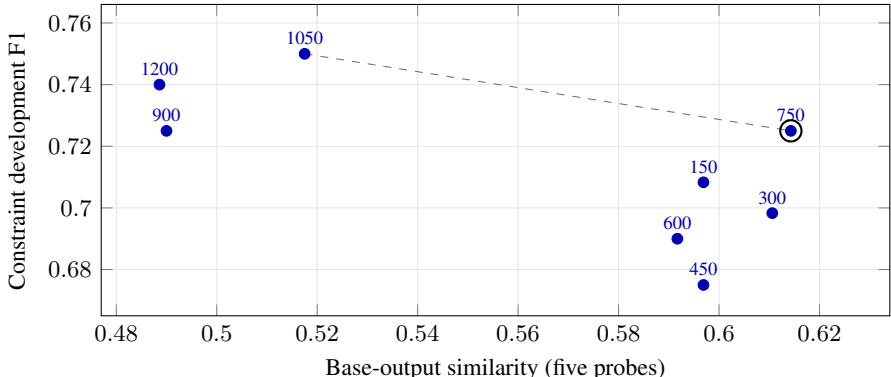


Figure 4: Constraint-F1/similarity projection with optimizer-step labels. The dashed segment connects the two nondominated points in this two-dimensional projection; it is not an interpolated achievable frontier. The circled point is selected step 750. Including QueryNER F1 produces the larger three-dimensional nondominated set reported in the main text. Similarity is not general-capability accuracy.

C Result-Level Provenance

Public code, models, and data. Readers can access the [GitHub project](#), [CommerceCore model card](#), [merged model weights](#), [model configuration](#), and [selected step-750 adapter](#). The upstream resources are the [Qwen3-1.7B base model](#), [QueryNER dataset card](#), and [QueryNER paper](#). Local dataset versions are linked as the [QueryNER training records](#), [QueryNER test records](#), [synthetic query targets](#), [catalog targets](#), and [20-query development set](#).

Fixed evidence versions. The GitHub evidence links in this appendix resolve to commit [fa64ee7](#); Hugging Face file links resolve to revision [425c4a5](#). These fixed versions prevent later edits from silently changing the cited evidence. The [archive manifest](#) records the full Hugging Face revision, file URLs, and SHA-256 hashes of the mirrored inputs. Current manuscript and audit links point to the maintained paper folder; the [build manifest](#) identifies the exact PDF and source-package contents.

Run the offline audit from the repository root with `python3 paper/audit_evidence.py`. Normalization uses Unicode case folding and whitespace collapse, without punctuation removal or semantic deduplication. The [reconstructed QueryNER sample IDs](#) are saved explicitly; the reconstructed prompt/completion sequence matches the archived mixed training file. The hash manifest identifies exactly which data and scripts were audited.

The implementation links are the [training loop](#), [query generator](#), [validator](#), and [archived serving parser](#). The eight-query [JSON-format smoke test](#) stores eight valid JSON outputs, which are independently parseable; it lacks gold constraints and is not an extraction-accuracy benchmark. The maintained [paper PDF](#), [LaTeX source](#), and [self-contained source ZIP](#) are linked for inspection and rebuilding.

Table 8 preserves the original manuscript’s additional benchmark values with their evidence status. A narrative claim or executable script alone is not a saved measurement. A comment also records a

Table 11: Clickable evidence map for the reported numbers. Offline rescoring verifies the stored predictions and arithmetic without rerunning model inference.

Result	Artifact and verification boundary
Mixture and seed-42 sample	Mixture builder ; archived 2,130-row mixture . Reconstructed records match in order.
75.18 minutes; \$0.9021	Run summary . Cost arithmetic verified; not a provider invoice.
Checkpoint trajectories	Task-F1 CSV ; similarity CSV ; selection note . Eight checkpoints.
Loss curve and epoch means	All 1,200 loss records ; training console log . Summaries and exact-step joins recomputed.
Served F1 and category scores	All 20 served predictions and labels ; evaluation script . F1 0.683 and vertical means reproduced.
Open-model development F1	Stored aggregates ; evaluation script . 0.050 and 0.100; predictions absent.
Full QueryNER F1 and CI	Result JSON ; scoring script . Historical per-example scores absent.
API QueryNER results	Result JSON ; prompt and evaluation script . Fifty queries, restricted labels.
Repeated API constraints	Result JSON ; evaluation script . Pooled-repeat CIs excluded from inference.
Single-run flagship results	Result JSON ; evaluation script . Kept separate from three-repeat runs.
Six-query local/API results	Local-model predictions ; API aggregates ; query definitions . Saved local predictions rescored.
Generation cost	Generation ledger . Sum of 1,630 call estimates verified.
Offline analyses and hashes	Audit results ; audit code . Includes archived-mixture checks and served-model rescoring.

313/450 generic-template issue in an earlier batch, but that batch is unavailable for recounting; no measured before/after quality improvement is inferred.

The original planning documents describe broader retrieval, tool execution, and merchant-transfer objectives. A plan is not evidence of a completed experiment. This paper reports the implemented extraction study only.

D Protocol for a Confirmatory Experimental Extension

This section specifies additional experiments motivated by the study, not completed results. The intended questions are the effect of task specialization, the contribution of each data source, robustness to linguistic and schema shifts, and accuracy–latency–cost trade-offs.

Data integrity and independent evaluation. Freeze an annotation contract for supported fields, negation, unit conversion, and unsupported requests. Serialize the validated target itself and audit completeness as well as span binding. Group repeated scenarios and paraphrases before splitting. Use development data for checkpoint and prompt selection. In a new training cycle, use the QueryNER development split rather than test examples. Do not relabel historical selection data as unseen. Build a separate constraint test set with documented provenance and label validation.

Robustness coverage. A planned 300–1,000-query benchmark should stratify one, two, three, and four-or-more constraints; negation; units; implicit values; paraphrases; and typos. Counts alone do not establish independence or label validity. Separate linguistically perturbed scenarios from merchant/schema transfer and include genuinely distinct scenarios or sources where permitted. Determine sample size from a prespecified precision or power target, not an observed favorable difference. Labels need independent validation; mechanically generated cases should be identified as synthetic stress tests.

Specialist baselines and ablations. Evaluate the unfine-tuned Qwen3-1.7B backbone, QueryNER-style BERT and domain-adapted BERT, a modern encoder tagger, a small sequence-to-sequence extractor, deterministic rules, API models, and CommerceCore on identical records. Supply the full ontology, appropriate templates, and sufficient generation budgets. Entity-only encoders should be compared on the entity task unless a documented normalization/extraction head makes constraint outputs comparable. Continued pretraining entails additional data and compute that must be reported.

Match presentation counts, token budgets, tuning allowance, and decoding for data-source and task-sharing ablations. “Real only” contains entity supervision but no synthetic constraint targets; that setting changes task coverage as well as data origin and should be interpreted accordingly. Table 12 defines the scope without inventing scores.

Table 12: Planned ablations; no results have been produced for this table. C = synthetic query constraints; E = human entity labels; A = synthetic catalog attributes. Historical checkpoint comparisons are not substitutes for these controls.

Configuration	Sources	Status	Interpretation
Real only	E	Not run	Entity training; constraint task lacks direct targets.
Synthetic only	C+A	Not run	Removes human entity supervision.
Real + query synthetic	E+C	Not run	Removes catalog task from the mixture.
No phrase gate	E+C+A	Not run	Requires archived pre-gate data and matched sampling.
No QueryNER multitask	C+A	Not run	Same data-source condition as synthetic only.
Corrected full setup	E+C+A	Not run	Repairs label integrity; historical results cannot be reused.

Inference and uncertainty. Run multiple training seeds and archive raw generations, parsed outputs, failure flags, per-query scores, model revisions, and settings. Prespecify a primary endpoint and paired query-level uncertainty; report exact-set correctness, precision/recall, and results by phenomenon and vertical. Use a scored general-capability suite for retention claims. Evaluate batch sizes 1, 8, 32, and 64 where memory permits, including sustained throughput, p50/p95 latency, full device memory, startup, utilization, and comparable API request costs. Record failures rather than omitting infeasible operating points.